

Discussion Claude Code — 18 août 2026

Archive complète de la session du 18/08/2026 — reprise après perte de session, investigation du déficit de zéros du run T=5M v16, sécurité, documentation. Compilée à partir des échanges VS Code de la journée. Les extraits de code modifiés ne sont pas reproduits ici ; seuls les tableaux, rapports, conclusions et les instructions transmises par hprzeta sont conservés intégralement.

1. Reprise de session — contexte initial (prompt de hprzeta)

REPRISE DE SESSION APRÈS PERTE — Riemann_Lab / PC1 (zeta-lab)

Session interrompue (Ctrl+Z puis fermeture de VS Code). Tu ne te souviens de rien. Voici l'état établi, ne le redécouvre pas.

CE QUI S'EST PASSÉ

Run T=5M v16 lancé 16/08 19h35 (PID 485255), intervalle T=[14, 5 000 000], PC1 seul (PC2 écarté : FLINT 2.9.0 vs 3.3.1 requis).

- 17/08 17h14 : run principal TERMINÉ. 10 016 297 / 10 016 474 zéros, 21h38 (1298,5 min), 128,56 z/s, 99,9% arb_C. Déficit : 177 zéros.
- 17h14 : rescan_segments_deficit() démarre — step/2 (0.000314 -> 0.000157) sur la LARGEUR COMPLÈTE des 8 segments, Pool.map() SYNCHRONE.
- 18/08 03:12 : PC1 s'éteint (arrêt propre, cause INCONNUE, journal volatil).
- Bilan : 31h37 perdues, RIEN récupérable. Cause racine = to_csv() placé APRÈS le rescan : le run principal était complet et valide en RAM à 17h14, jamais flushé.

Anomalie annexe non investiguée : cascade de 6 reboots courts entre 06:19 et 06:30 le 18/08.

Précédent v13 (T=5M, 27-29/06) : même rescan intégral = 18,8h pour +1 zéro net sur 96 manquants. Rendement ~1%.

CE QUE JE TE DEMANDE MAINTENANT — Étape 0 (point projet + vérification git/Handoff/JOURNAL), Étape 1 (Bloc B — fiabilité PC1 pour un run de 21h : journal persistant, apt history, smartctl, upower, dmesg), Étape 2 (Bloc A — préparer les diffs de persistance CSV sans les appliquer), Étape 3 (Bloc C — test v15/v16 sur T=100k pour trancher la régression de détection).

2. Étape 0 — Point projet : écarts détectés

Alerte sécurité immédiate : docs/archive/zeta-secure-backup.tgz (non suivi) contenait etc/wireguard/privatekey — une clé privée WireGuard en clair dans le repo de code.

Écart majeur Handoff/JOURNAL vs code réel : Handoff.md (figé au 17/08 journée) et JOURNAL.md (figé au 16/08) ne mentionnaient ni la fin du run, ni le crash, ni les correctifs — alors que les commits 2f2047a (persistance CSV avant rescan + checkpoint par worker + --skip-rescan) et a391beb (pointeur v16 + systemd-inhibit) étaient **déjà committés** sur Riemann_Lab_C, datés du matin même (08h59 et 10h37). Le Bloc A demandé "à préparer sans appliquer" était donc déjà fait.

Git : branche Riemann_Lab_C propre à part scripts/zeta_run_progress.py modifié (fix dashboard ETA, non commité) et 2 fichiers de suivi locaux non trackés.

3. Étape 1 — Bloc B : PC1 est-il fiable pour un run de 21h ?

Vérification	Résultat
Règle sudoers zeta_diag	N'existait pas — décision prise de ne pas la créer
Journal persistant	/var/log/journal/<machine-id>/ n'existait pas avant le boot du 18/08 08h19 — journal no
journalctl --list-boots	Un seul boot listé (18/08 08h19) — aucune trace du crash 03h12 ni de la cascade 06h
/var/log/apt/history.log	Dernière MAJ 15/08 21h04-21h21 (firmware, pas de noyau) — 3 jours avant le crash
upower (batterie)	98%, fully-charged, 753 cycles, capacité 70,2% — usure notable
smartctl -a /dev/sda	SMART overall-health : PASSED, 0 secteur réalloué, 0 secteur en attente, 0 erreur non corri
dmesg	Aucune ligne thermal/mce/hardware/critical — mais tampon volatile par boot, mêm

Verdict : INDÉTERMINÉ pour la cause du crash, mais **disque exclu** (SMART propre).
Candidat le plus plausible : batterie usée.

4. Priorité 1 — Sécurité (clé WireGuard)

- Vérifié : docs/archive/zeta-secure-backup.tgz contenait bien etc/wireguard/privatekey en clair.
- **Recherche exhaustive dans tout l'historique git** (tous les blobs, toutes les branches) : **aucune trace, jamais commité** — pas de filter-repo nécessaire.
- Archive déplacée vers ~/riemann_handoff/secrets_local/ (hors repo).
- .gitignore de Riemann_Lab_C mis à jour (docs/archive/zeta-secure-backup.tgz + docs/archive/*-backup.tgz), testé et fonctionnel — **pas encore propagé aux 3 autres branches**.
- **Reste à faire (action hprzeta, hors Claude Code)** : régénérer la clé WireGuard PC4 et le token DuckDNS par précaution.

5. Priorité 2 — Resynchronisation de la documentation

- ~/riemann_handoff/Handoff.md : nouveau bloc daté du 18/08 inséré en tête (run terminé/perdu, correctifs, Bloc B indéterminé, sécurité), historique conservé en dessous.

- Wiki JOURNAL.md : nouvelle entrée "2026-08-18, 12h00" insérée en tête (convention append-only, plus récent en haut) — bilan complet du run T=5M v16, cause racine, correctifs 2f2047a/a391beb.
- scripts/zeta_run_progress.py (fix dashboard ETA — mode solo détecté automatiquement, ETA pondérée en $\sqrt{t}$) commité seul : 52ba8d9.
- Wiki non commité/poussé à ce stade (repo distinct, en attente de validation).

6. Priorité 3 — Bloc C : investigation du déficit de zéros

6.1 Test v15 vs v16 sur T=100 000

Hypothèse initiale : acb_dirichlet_hardy_z en précision fixe 64 bits (v16) rate des paires de zéros proches là où arb_fpwrap adaptatif (v15) montait en précision automatiquement.

Résultat : v15 et v16 produisent des CSV **strictement identiques** — 138 069 zéros chacun, écart maximal 0,000e+00 sur toute la liste. Les deux runs sont individuellement COMPLET côté Turing-Backlund. **Hypothèse réfutée** à cette échelle.

6.2 Bug corrigé au passage : zeta_turbo_off.sh

zeta_turbo_on.sh arrête 12 services (dont snapd, fwupd), mais zeta_turbo_off.sh n'en redémarrait que 4 — snapd/fwupd restaient arrêtés après chaque run. Liste des services synchronisée entre les deux scripts (non commité).

6.3 Question de code — que fait MARGE_SECURITE ?

Réponse (lecture de code, v13 et v16 identiques) :

$$\text{STEP}(T) = \text{KAPPA} \cdot \text{gap_moyen}(T) \cdot N(T)^{(-1/3)} / \text{MARGE_SECURITE}$$

Calculée **une seule fois** par run, sur le τ_{MAX} global, puis appliquée uniformément à tout le run. Ne touche ni les bornes de segments, ni le nombre de termes Riemann-Siegel, ni la précision d'évaluation — uniquement la densité de la grille de scan.

Correction factuelle : v13 réel utilisait MARGE_SECURITE=2.0 (commit a0e6e41, 04/07, message : "Run T=5M terminé : 96 manquants... MARGE=2.0"), pas 3.0 comme supposé initialement. Contraste réel v13→v16 : $\times 5$ sur STEP, pas $\times 3,33$.

6.4 Test A/B — MARGE=2.0 vs MARGE=10.0 sur [4 900 000 ; 5 000 000]

	MARGE=2.0	MARGE=10.0
Zéros trouvés	216 081	216 081
Durée	9,6 min	40,2 min
STEP	0,0015712	0,00031424

Comparaison zéro à zéro : identique, écart max $1,35\text{e-}7$ (bruit numérique d'affinage), 0 paire divergente au-delà de $1\text{e-}6$.

Analyse d'espacement ($s = g/\delta(t)$, $\delta(t) = 2\pi/\ln(t/2\pi)$) : espacement minimal trouvé $s_{\text{min}} = 0,0248$ (soit un écart brut $\approx 0,0115$), confortablement au-dessus des deux STEP testés. **Aucun espacement $< 0,010$** dans les deux runs. s moyen = 1,0000 (conforme GUE). **La fenêtre**

testée ne contient aucune paire assez serrée pour mettre en tension l'une ou l'autre grille — et la zone à risque identifiée le 02/08 ($T \approx 4,86M$) est hors de cette fenêtre.

6.5 Test A/B — MARGE=2.0 vs MARGE=10.0 sur [4 800 000 ; 4 900 000] (inclut $T \approx 4,86M$)

	MARGE=2.0	MARGE=10.0
Zéros trouvés	215 756	215 756
Durée	9,5 min	38,9 min
STEP	0,0015852	0,00031704

Comparaison zéro à zéro : identique, y compris dans la zone précise $T \approx 4\ 862\ 705$. Un espacement anormal de 1,538 unité ($s \approx 3,32$) repéré entre 4862705.039 et 4862706.578, identique dans les deux runs — vérifié par un scan indépendant $5\times$ plus fin (arb_hardy_z, pas le pipeline normal) : **exactement les 4 mêmes zéros retrouvés, rien de caché**. Ce n'est pas une paire manquée, juste une grande fluctuation statistique.

Bilan des deux fenêtres (200k au total) : 0 différence entre MARGE=2.0 et MARGE=10.0, partout, y compris là où le récit du 02/08 affirmait un échec de marge plus faible.

6.6 Calcul théorique GUE (prompt de hprzeta) — ferme la piste résolution

Nombre de paires enjambées attendu (loi GUE des petits écarts) : $M \approx (\pi^2 \cdot STEP^3)/(9 \cdot (2\pi)^4) \cdot \int L(t)^4 dt$, $L(t) = \ln(t/2\pi)$, $\int \approx 1,20e11$ sur [14, 5M]

- STEP = 0,000314 (v16, MARGE=10) $\rightarrow M \approx 0,003$ zéro
- STEP = 0,00157 (v13, MARGE=2) $\rightarrow M \approx 0,33$ zéro

Observé : 177 et 96. Soit 5×10^4 fois et 3×10^2 fois la prédiction. Conclusion : NI les 177 NI les 96 ne sont des zéros manqués par la grille. Les deux déficits ont une autre origine, commune aux deux versions. Corollaire : l'explication "paires de zéros proches dont les signes se compensent" inscrite au JOURNAL depuis v13 est FAUSSE. À corriger.

6.7 Lecture de code approfondie (6 points demandés par hprzeta)

1. Formule STEP — confirmée constante sur tout le run, valeurs exactes vérifiées bit pour bit contre les logs réels (0,00031424140619485785 pour MARGE=10, 0,0015712070309742891 pour MARGE=2). La mention STEP=0.010 fixe du STACK.md est un vestige historique (table de versions retirées v1/v3/v6), la chaîne "ne jamais modifier" n'existe nulle part dans ce fichier.

2. Formule N_ATTENDU — deux formules distinctes et incompatibles dans le projet :

- turing_validation.N_exact(T) : $\lfloor \Theta(T)/\pi \rfloor + 1 + \text{round}(S(T))$ — précise, utilisée par la validation finale.
- _n_zeros_expected(T) : $\text{int}(T/(2\pi) \cdot \ln(T/(2\pi e)))$ — Weyl brut, sans $\Theta(T)$ ni $S(T)$, utilisée uniquement en interne pour déclencher le rescan par segment.

Le "déficit" du rescan et le "manquants" de Turing **ne sont pas calculés avec la même formule**.

3. Bornes de segments — scan réel avec chevauchement de 0,5 unité par sécurité ; comptage/déficit en intervalles demi-ouverts non chevauchants ; déduplication (tolerance=0.01) appliquée sur la fusion des 8 workers avant le calcul du déficit.

4. Distribution du déficit par segment — tableau mesuré sur les logs réels :

Segment	Bornes (t)	v13 (réel, validé)	v16 (réel, non validé)
0	[14, 737 112]	+18	+51
1	[737 112, 1 391 397]	+9	+49
2	[1 391 397, 2 020 448]	+10	+16
3	[2 020 448, 2 634 120]	+13	+13
4	[2 634 120, 3 236 822]	+10	+10
5	[3 236 822, 3 831 051]	+9	+9
6	[3 831 051, 4 418 407]	+14	+14
7	[4 418 407, 5 000 000]	+14	+14
Total		97 (→96 après rescan)	176-177 (jamais rescanné)

Toute l'augmentation du déficit est concentrée à plus de 90% dans les segments 0-1 (bas T) — signature incompatible avec un mécanisme dépendant de T croissant.

5. Turing-Backlund — recherche exhaustive de "VALIDATION TURING-BACKLUND" dans les 563 953 lignes du log réel du run v16 : **0 occurrence**. Le run a été tué par le crash exactement au point *"8 segment(s) en déficit → rescan..."*, avant d'atteindre la validation exacte. **Le "177" n'a jamais été validé par la méthode de référence du projet**. Pour v13, `valider_turing()` a bien tourné jusqu'au bout : verdict réel × INCOMPLET, 96 manquants confirmés par la méthode exacte.

6. Diff v13 vs v16 (comptage/segmentation/fusion) — diff strictement vide sur `_partitionner_adaptatif` ; fusion des workers identique à deux ajouts fonctionnellement inertes près (checkpoint CSV, `pool.map→imap_unordered`). **Réserve non testée** : le détecteur C partagé `scan_arb.c` a été modifié par le commit `d4b3611` (cache `log_n/isqrt_n`, v14) entre le run v13 (avant) et le run v16 (après) — jamais comparé binaire contre binaire.

6.8 Correction JOURNAL (récit du 02/08 invalidé)

Entrée insérée dans `JOURNAL.md` (18/08, 18h00) : le récit du 02/08 concluant que `MARGE_SECURITE=3.0` échouait à $T \approx 4,86M$ et que `MARGE=10.0` réglait le problème est invalidé par les tests A/B directs du 18/08 (résultats identiques dans les deux configurations, y compris dans la zone exacte visée). Le calcul GUE ferme définitivement la piste "résolution de grille". Concentration du déficit sur les segments 0-1, incompatible avec un mécanisme dépendant de T.

6.9 Tentative "CSV_principal du run v16" — corrigée

Suggestion initiale erronée : le CSV `_principal` (mécanisme introduit par le commit `2f2047a`, après l'incident) n'existait pas au moment du crash réel — le dossier `calculs/v16_T5000000_20260816_193618/` est vide. Le run T=5M v16 réel reste définitivement **non vérifiable** : ni ses zéros ni son verdict Turing ne pourront jamais être obtenus a posteriori. Correction insérée dans le JOURNAL.

6.10 Validation Turing exacte, correctement windowée

Faute de CSV du run réel, application de la formule exacte ($N_{exact}(T) = \lfloor \Theta(T)/\pi \rfloor + 1 + \text{round}(S(T))$) aux CSV du test A/B déjà en main (`[4,8M;5M]`, 431 837 zéros) :

Fenêtre	Calculés	Attendus (exact Θ +S)	Déficit
[4 800 000, 4 900 000)	215 756	215 760	+4
[4 900 000, 5 000 000)	216 081	216 085	+4

Fenêtre	Calculés	Attendus (exact $\theta+S$)	Déficit
Total	431 837	431 845	+8

Contrôle de robustesse : $s(\tau)$ recalculé avec $n_sigma = 50/100/200/400$ et $dps = 35/50$ — valeur strictement identique dans tous les cas ($s(4\ 900\ 000) = -0,3091$ invariant). Le déficit $+4/+4$ n'est pas un artefact numérique.

Interprétation : ce taux ($\sim 1,9 \times 10^{-5}$) est du même ordre de grandeur que le déficit grossier du segment 7 ($\sim 1,1 \times 10^{-5}$) — cohérent avec un **déficit de fond à haut T, indépendant de MARGE_SECURITE**, identique entre v13 et v16 sur cette zone. **Ce déficit de fond n'explique probablement pas l'écart 96→177**, qui reste concentré dans les segments 0-1, jamais testés par cette méthode. Résultat inséré dans le JOURNAL (18/08, 19h00).

6.11 Test en cours — segments 0-1 (bas T)

Lancement d'un test A/B identique (MARGE=2.0 puis MARGE=10.0) sur [14 ; 1 391 397,1] — segments 0 et 1 combinés, là où la vraie anomalie (déficit $\times 2,8$ et $\times 5,4$ entre v13 et v16) a été mesurée. STEP calibré sur le véritable τ_{MAX} global de production (5 000 000), pas sur la borne locale du test, pour reproduire exactement les conditions réelles. Segmentation calculée : 8 workers de largeur 160 577 à 208 945 unités. Estimation : ~ 40 -50 min (MARGE=2.0), $\sim 2,5$ -3h (MARGE=10.0). **Run en cours au moment de la rédaction de ce document** — résultat à suivre dans une prochaine entrée JOURNAL.

7. Points ouverts en fin de journée

- Bloc B (fiabilité PC1) : INDÉTERMINÉ, disque exclu.
- Écart 96 vs 177 : origine réelle toujours inconnue, concentrée sur les segments 0-1 ; piste ouverte non testée : différence de binaire `scan_arb.c` (commit `d4b3611`) entre les deux runs.
- Commits en attente : `.gitignore` (clé WireGuard), `zeta_turbo_off.sh` (services manquants).
- Wiki JOURNAL.md : 3 nouvelles entrées insérées, non commitées/poussées.
- Propagation du `.gitignore` corrigé aux 3 autres branches.
- Régénération de la clé WireGuard PC4 et du token DuckDNS (action `hprzeta`).
- Suivi cron `rsync/rc1one` PC3 non reconfirmé depuis le 17/08.
- Test A/B segments 0-1 en cours au moment de la rédaction.

Document généré à la demande de hprzeta le 18/08/2026 — archive de session, Claude Code — Riemann_Lab, branche Riemann_Lab_C.